Concurrency and Synchronization

Deadlock and Starvation

- Deadlock Characterization
  - Four necessary conditions: Mutual exclusion, hold-and-wait, no preemption, circular wait
    - Example: Processes each hold resource, request next in cycle
  - Starvation: Process indefinitely denied resource despite availability (e.g., priority inversion)
- Deadlock Prevention and Avoidance
  - Prevention: Deny one condition (e.g., request all resources at start, allow preemption)
    - Drawbacks: Low utilization, deadlock-free but inefficient
  - Avoidance: Banker's algorithm checks safe state before allocation
    - Tracks max claim, available, allocation; simulates to ensure progress
- Deadlock Detection and Recovery
  - Detection: Resource allocation graph, look for cycles (single/multiple instances)
    - Periodic run or on timeout
  - Recovery: Terminate process, preempt resources, rollback
    - Choose victim by cost (priority, resources)
- Dining Philosophers Problem
  - Five philosophers, shared forks; need two to eat
    - Deadlock risk: All pickup left fork, wait for right
  - Solutions
    - Resource hierarchy: Number forks, always pick lower first
    - Chandy/Misra: Odd pick left/right first, even reverse; request from neighbors
    - Arbitrator: One fork managed centrally

Basic Principles

- Critical Sections and Race Conditions
  - Race conditions occur when multiple threads access shared data concurrently without proper synchronization, leading to unpredictable outcomes
    - Example: Two threads incrementing a counter simultaneously may result in lost updates
    - Requirements for solutions: Mutual exclusion, progress, bounded waiting
  - Critical section problem: Code segment where shared resources are accessed, must ensure only one thread executes it at a time
    - Entry section: Code to acquire access
    - Exit section: Code to release access
    - Remainder section: Non-critical code
- Synchronization Goals
  - Mutual exclusion: Ensures only one process/thread in critical section
  - Progress: If no one in critical section, decision on entry must be finite
  - Bounded waiting: Limits time a process waits for entry, prevents starvation

Synchronization Mechanisms: Locks

- Basic Idea of Locks
  - Locks provide mutual exclusion through acquire/release operations
    - Acquire: Block until lock held by no one, then take it
    - Release: Allow another to acquire
  - Busy-waiting (spinning): Thread repeatedly checks lock availability, wastes CPU
- Spin Locks with Test-And-Set (TAS)
  - TAS atomic instruction: Reads word into register, writes all 1s if originally 0 (returns original value)
    - Pseudocode: while (TestAndSet(lock)); // spin until acquire
    - Building spinlock: Initialize lock=0, use TAS for atomic acquire
  - Advantages: Simple, works on uniprocessors/multiprocessors
    - Disadvantages: High CPU overhead on contention, poor for long holds
- Compare and Swap (CAS)
  - CAS atomic: Compares memory value to expected, swaps with new if match
    - Pseudocode: CAS(address, expected, new) returns true if swapped
  - Non-blocking synchronization: Optimistic, retries only on conflict
    - Used in lock-free data structures like queues

Using Queues: Sleeping Instead of Spinning

- Blocking Synchronization
  - Replace spinning with sleep/wakeup: Thread yields CPU when lock unavailable
    - Scheduler queues waiting threads, wakes on release
  - Benefits: Saves CPU cycles, fairer on multiprocessors
    - Implemented via condition variables or event queues
- Mesa vs Hoare Monitors
  - Mesa style: Wakeup may not immediately grant lock, possible missed signals
  - Hoare style: Wakes thread, transfers lock immediately (complex)

Case Study: Linux Kernel Synchronization

- Spin Locks
  - Busy-wait locks for short critical sections, disable interrupts on UP
    - Use on multiprocessors: Per-CPU spinlocks reduce cache contention
    - Implement: Atomic TAS or ticket locks for fairness
- Semaphores
  - Counting semaphores for resource counting, sleep on zero
    - down() = P, up() = V, with wait queues for blocking
  - Used for device drivers, complex producer-consumer
- Mutexes
  - Optimized for sleeping, priority inheritance to avoid inversion
    - Faster than semaphores for simple mutual exclusion
    - States: unlocked, locked contested/uncontested, with owner tracking

Semaphores

- Definition and Operations
  - Semaphore: Non-negative integer variable with atomic P (wait) / V (signal) ops
    - P(s): if s>0 decrement else block; V(s): increment, wakeup waiter if any
  - Counting vs binary: Counting for resource pools, binary (0/1) as mutex
- Binary Semaphores
  - Acts like lock: Init to 1, P acquire, V release
    - Simpler than locks in some contexts, supports priority inheritance
- Producer/Consumer (Bounded Buffer) Problem
  - Shared buffer with n slots, producers add, consumers remove
    - Semaphores: mutex (buffer access=1), empty (n), full (0)
  - Solution sequence:
    - Producer: P(empty), P(mutex), add item, V(mutex), V(full)
    - Consumer: P(full), P(mutex), remove item, V(mutex), V(empty)
    - Ensures no overflow/underflow, mutual exclusion
- Reader-Writer Locks
  - Multiple readers allowed simultaneously, writers exclusive
    - Semaphores: mutex (1), wrt (1), rdcnt (read count protector)
  - First/second readers/writers protocol:
    - Reader: P(mutex), if first increment rdcnt P(wrt), else just V(mutex); read; symmetric exit
    - Handles fairness issues, writer starvation possible in reader-favoring version

Presented with xmind

SUBMITTED BY

Group 2

ATP24CS002 – AADITH K S
ATP24CS006 – ABHINAND SHAJI K
ATP24CS010 – ADHIL K A
ATP24CS014 – AFTHAB RAHMAN A
ATP24CS018 – AKSHARA ANILKUMAR
ATP24CS022 – AMALDEV R
ATP24CS026 – ANGEL RONIYA PETER
ATP24CS030 – ANURUDH P
ATP24CS034 – ARROW THOMAS M T
ATP24CS038 – ASWIN M P
ATP24CS042 – BIBIN KRISHNA S
ATP24CS046 – DHANIKA H
ATP24CS050 – DRISYA C
ATP24CS054 – HARSHJ K V
ATP24CS058 – JESNA J